The Discovery Engine

A map of the derive-first discovery cycle:
the flow, the artifacts, the gates, and what is real vs. authored vs. derived.

discovery-engine **0.6.0** (`main`)

2026-11-11

Derive-first. One pure projection over an authored substrate.
One measured input. The simulation run (`runs.db`).
The loop closes. Refutation → next study; verdict → discovery; feedback → reopen.

1 The idea in one page

The discovery engine turns an **investigation** — an `investigation.yaml`, its member `study.yaml` files, and each study's registered runs — into a reproducible view of *what is known* and *what to do next*, and drives the cycle that advances it. Two properties hold everywhere.

- **Derive-first.** Every view is *computed* from the studies, then merged over any authored override. The rule is uniform: **authored wins, derived fills; an empty authored value never clobbers a derived one.** Same inputs → same bundle.
- **Pure core, I/O at the edge.** The projection and the phase functions are pure (dicts in, dicts out). Only the execute phase and a few explicit writers touch disk.

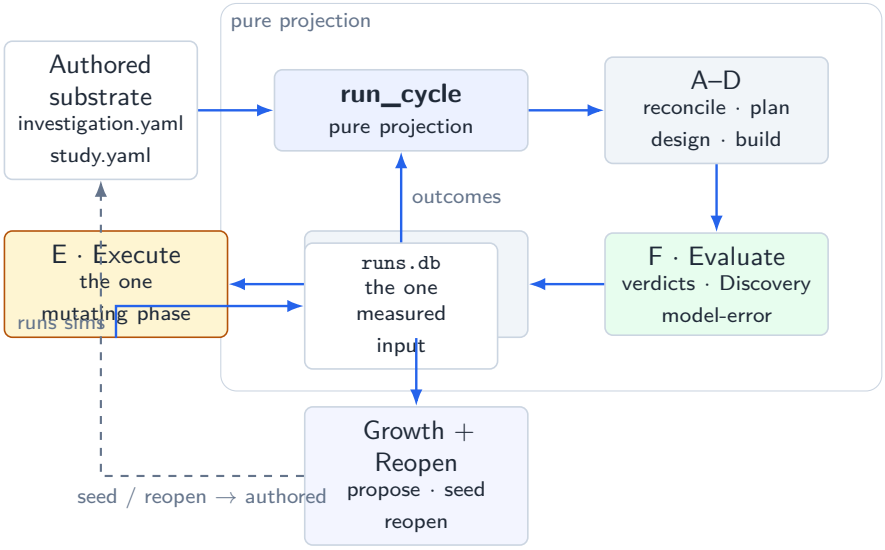
The engine **performs no science itself**. It reconciles, plans, designs, routes model acquisition, evaluates, and composes; the actual simulation runs happen in the workspace (via vivarium-workbench) and the engine reads their results back. The **one genuinely measured input** is a run's pass/fail outcomes — everything else (verdicts, edges, ordering, gaps, gates, actions, designs, proposals, discoveries) is derived.

The loop now closes. Earlier revisions could only *advance* an investigation toward closure. As of 0.4.0 the cycle also *regenerates* itself and *re-enters*:

- a **refuted** finding proposes the next study (a builds-on child seeded from the finding, plus a synthesis study folding the refutations into the next discriminating question);
- a **model-error diagnosis** decides *which* child to seed — revise the model or revise the hypothesis;
- a claim verdict that flips to a *settled* outcome on *verified* evidence records a first-class **Discovery**;
- staged scientist feedback re-enters a closed investigation through the **reopen** op — a policy-gated cycle move, not a side channel.

Self-contained, and evidence versioned. Two later increments harden this. The engine became **self-contained** (0.4.1–0.5.0): the report render, the report lint, the study-maintenance verbs, and the whole linkage index moved out of the workbench and the skills into engine-native `discover` commands that run *headless*, with no server. And the evaluation snapshot became **versioned** (0.6.0): it carries its grader/resolver versions and a hash of the measured evidence, so a verdict flip diffed against a provenance-incompatible snapshot is re-baselined, not mined as a false Discovery.

2 The cycle at a glance



The engine has **no single function that chains its phases**. `run_cycle` is the pure core; the lettered phases are separate functions layered on its output, each also a `discover` CLI verb. The one canonical ordered step graph lives in the workbench panel; **`run_cycle` is pure**, and state changes only through the execute phase and a few explicit write-backs.

The steps. Fourteen in the panel's step graph — thirteen pure, one mutating.

Step	Function	Role	Kind
Inputs	`load_cycle_inputs`	load the substrate	pure
Core	`run_cycle`	project the whole cycle	pure
A · Reconcile	`reconcile_evidence_state`	grade the evidence	pure
B · Plan	`plan_candidate_actions`	rank the moves	pure
C · Design	`design_discriminating_study`	design the test	pure
D · Build	`build_catalog` / build-model	acquire the model	pure
F · Evaluate	`compare_hypotheses`	verdicts + Discovery	pure
Grading	`compute_outcomes`	the sync that feeds the verdict	pure
Growth	`suggest_studies`	propose the next studies	pure
· select	`select_for_launch`	the autonomy gate	pure
· seed	`build_seeded_study`	author a run-ready study	pure
· refine	`refine_study`	deepen an existing study	pure
Reopen	`reconcile_and_reopen`	re-enter on feedback	pure[†]
E · Execute	`advance_cycle`	run + change state	**mut.**

[†] `reconcile_and_reopen` is a pure planner; its write happens through the execute phase / the `ingest-feedback` writer. The one mutating step (`advance_cycle`) runs sandboxed in the panel — it copies the investigation into a throwaway workspace first, so a cycle run never touches real files.

3 The data substrate — what the cycle reads and writes

The cycle operates on a **workspace** directory whose layout is resolved from `workspace.yaml` (never hard-coded). Three artifact classes:

- `investigation.yaml` — the investigation spec. **Authored:** `question`, `audience`, `studies[]`, `hypotheses[]`, `acceptance_criteria[]`, `plan_gates[]`, `required_mechanisms[]`, `cycle_flags`, `decisions_needed`, epistemic `relation` tags on edges. **Cycle-written caches:** `resolutions[]`, `discoveries[]`, `applied_feedback[]`, `lifecycle_history`, `cycle_state` (with `evaluation_snapshot`), proposal caches.
- `study.yaml` — one per member. **Authored:** `claim`, `title`, `question`, `hypothesis`, `behavior_tests` (with `pass_if` bands + `measure`), `findings[]` (`evidence.observed`, `from_test`), `baseline`/variant arms, `observables`, `required_mechanisms`, `parent_studies` + relations. The `runs[]` list is **code-synced** from `runs.db`.
- `runs.db` — SQLite, written by the simulation machinery. **The durable record of what actually ran** — the one measured artifact.

Provenance. Throughout this map: **(H)** human-authored, **(AI)** agent-authored (via the `/viva-*` skills), **(D)** derived/computed from evidence, **(CFG)** config / schema / rollout flag. The verdict/gate/acceptance roll-ups read the *computed* outcome, not the authored one: a test that *computes* fail can no longer read "met" through a gate.

4 The stages

Each stage: **what happens** → **consumes** → **produces** → **is it real?** All stages except Execute are pure and re-runnable.

Inputs — load_cycle_inputs *pure reader*

Resolves `investigations/<slug>/investigation.yaml`, builds the **member set** from the spec's own `studies:`/`members:` (member-scoped — an undeclared membership can never pull a sibling investigation's studies into the cycle), and loads each `study.yaml`. Produces `(inv_path, inv_spec, {slug: study_spec})`; writes nothing. **real**.

Core — run_cycle *pure, no I/O*

Projects one full cycle under the merge rule. **Consumes** the authored knowledge/argument (H/AI) merged over derived (D), per-study `runs[].outcomes` and `behavior_tests` (D on outcomes, H on tests), epistemic `relation` edges (H), and the optional evidence/viz state. **Produces the bundle:** `knowledge`, `argument`, `narrative` (a deterministic topological ordering of claims — *not* generated prose), `audit`, `gates`, `open_calls`, `validations`, `suggestions`, `signals` (model disagreements, anomalies), and the viz/revision proposals. The validators leave `scientific_judgment` null — **none asserts scientific truth**. **real** (mechanical; quality is a function of authoring).

A — reconcile_evidence_state *pure*

Classifies each study **finding** by provenance into `verified` / `provisional` / `contradicted` / `unsupported`, tallies a summary, and rolls each study up by its *worst* finding. **VERIFIED** requires that the cited run *completed* and that `observed` matches the run's *computed* value within tolerance — not merely that a run id is registered. A study with no `runs.db` can never be verified. **real** — the one contract (`registered_run_contract`) for "what makes a finding run-backed."

B — plan_candidate_actions + select_portfolio *pure*

Recasts the frontier into classified CandidateActions (REPAIR / RESOLVE / EXTEND) with a heuristic value score; `select_portfolio` orders them (repair prereqs first, then resolve by discrimination, then extend). **real**, entirely frontier-driven — with no hypotheses nothing is discriminable, so the plan is all repair/extend.

C — design_discriminating_study *pure*

Returns `None` unless a study has a *discriminable* hypothesis set (≥ 2 live hypotheses predicting the same observable with **disjoint numeric bands**). Otherwise packages the authored hypotheses into an ExperimentDesign (one arm per prediction, control arms by naming heuristic, a pre-committed decision rule). **conditional** — all scientific content is authored; the function only reformats + adds control detection.

D — build_catalog / build-model *pure*

Walks every study's baseline + variant arms, groups composites by path, and builds one "component" per composite with a token-based capability signature. Resolution (*reuse* / *wrap* / *build*) runs on the evidence path when a workspace root is passed — that route is what *sources* a capability gap. A required mechanism with no registered synonym is matched by honest token coverage, so a novel compound routes to **BUILD** and the gap fires. In autonomous advance the `needs_build` route now resolves into a concrete immutable **ModelVersion** build (content-addressed, parent-linked; never overwrite the model that produced prior evidence). **real**.

F — compare_hypotheses + Discovery + model-error *pure*

Evaluate. Per study, recomputes each hypothesis's support log from a quantitative source (an observed value matching a prediction's band) and picks a `favored` hypothesis and a status: `supported` (net>0 *and* VERIFIED evidence), `provisionally-supported`, `contested` (net<0), `unresolved`, or `gated`. `model_disagreement_map` surfaces multi-arm disagreement on the same observable (*never averaged*); an *adjudicated* supersession is resolved history, not a live caveat.
Discovery record (0.4.0–0.6.0). `detect_discoveries` compares the current `evaluation_snapshot` against the previous one and records a first-class **Discovery** (scope, novelty, counterevidence, evidence refs) for every claim that *flipped to a settled outcome* (`supported`/`contested`) **on VERIFIED evidence**. Passing a gate is not a discovery; a flip on merely provisional evidence is not one either. **(0.6.0)** the snapshot now carries its own provenance — a `schema_version`, the `grader`/`resolver` versions, and an `inputs_hash` over the measured ledger — and a flip diffed against a snapshot from a *different* grader/resolver or different measured inputs is `stale`: re-baselined, not recorded, so a Discovery reflects new science rather than a grader bump or shifted run data.
Model-error diagnosis (0.4.0). On a refutation, `diagnose_refutation` emits a typed **Evidence-Update**: `revise-model` when competing model arms disagree on the refuting observable (the model choice is implicated), else `revise-hypothesis` (the model is coherent; the prediction was wrong). The diagnosis steers *which* child seed the Growth phase proposes. **real**.

Grading — compute_outcomes *the sync that feeds the verdict*

The reconcile sync that writes each run's `computed_outcomes` block, which the roll-ups read *computed-first*. For each recorded run it grades every `behavior_test` against the run's store and writes

`{result, measured_value, evaluated_by, ...}`. A bare-string measure is graded through the shared `viva_emitters` resolver (the engine grades byte-identically to the workbench); a missing store degrades to the agent / needs-rerun bucket rather than crashing. **This is the one path that reads run data** — it is what makes a refutation (and therefore the generative loop) fire on a *computed* result, not an authored label. **real**.

Growth — propose → select → seed → refine *pure; the generative loop*

Propose (`suggest_studies`) de-duplicates proposals from five sources: authored followup proposals (H/AI), narrative-gap bridges from the audit (D), `decisions_needed` that name a member study (H, gated on a decision), and — the **generative loop** — a *refuted* finding auto-spawns (4) a builds-on follow-up seeded from the finding and (5) a synthesis study folding the refutations into the next discriminating question. A refutation is a concrete signal, so these are proposed `ready` and flow through the autonomy gate. **Select** (`select_for_launch`) is a thin shim over the one `AutonomyPolicy` (below): in `autonomous` mode every ready proposal launches; in default `human_in_the_loop` only approved, non-expert-gated proposals do. **Seed** (`build_seeded_study`) authors a complete, run-ready `study.yaml` — it inherits the parent's composite, observables and behavior tests (so the child is runnable and evaluable by construction) and applies the proposal's `model_change` as the variation under test. **Refine** (`refine_study`) deepens an existing study from the engine's own signals (replicate count, low-fidelity flags). **real** — it proposes *where to look next* from a negative result, authors the child, and (under the autonomy policy) promotes it to run.

Reopen — reconcile_and_reopen *pure planner; the re-entry*

Staged scientist feedback — a compiled request on the investigation's `pending_feedback[]` — is a first-class cycle move. It surfaces as a `pending_feedback` frontier item whose op is **reopen**; the execute phase applies it under the autonomy policy, re-opening the closed investigation (status + verdict + `lifecycle_history`) and recording the applied id in `applied_feedback[]` so it is not re-applied. `ingest-feedback` compiles a scientist's reopening YAML into engine-native authored artifacts (typed actions, validated against the schema; no prose inference). **real** — the structured on-ramp by which an investigation *re-enters* the cycle after a verdict.

E — advance_cycle *mutating*

The phase that changes real state. Loop: reload → `execute_plan` → pick the first run-ready advance op not yet seen → check readiness → dispatch (or, if not autonomous, record one "would-run" and stop). Its dispatch set widened as the loop closed: besides running a study (`test`/`branch`) it now also **promotes a seed** to a real study, **builds a missing mechanism**, and **reopens** on staged feedback. Every consequential dispatch is checked against **one** `AutonomyPolicy` (0.4.0 unified it from per-op gates). **real** but **dry by default**: in the panel it runs *sandboxed* on a throwaway copy, never on real files; with no runner it reports `needs_build` rather than faking a run.

Supporting derivations. Three non-blocking families ride the same projection and feed the frontier: the **viz loop** (declared figures classified fresh / unrendered / stale / broken, with typed proposals for uncovered gates, discriminations, and disagreements); the **capability-gap bridge** (a required mechanism no catalog component provides → a build-or-decline gap); and **narrative / viz-appropriateness freshness** (is a stamp stale, is a figure shown where it belongs). They are advisory — they inform the frontier, they do not gate closure.

5 The gate system — one engine, four views

A single function `frontier()` builds the list of unresolved items; `closure()`, `next_action()`, `opportunities()` and `execute_plan()` are re-rankings or filters over it. All pure and deterministic.

frontier() — the one list everything reads

An item is *open* if a source raises it **and** its `(kind, target)` is not in the `resolutions` set (the single discharge test). Each item carries a `priority` and an `op`; the list is sorted by `(-priority, kind, target)`. **Closure blocks on only three kinds** (shaded); every other kind is "open but non-blocking."

pri	kind	op	blocks?
90	unaddressed_criterion	branch	**blocks**
80	open_challenge	synthesize	**blocks**
70	narrative_gap	branch	**blocks**
60	open_decision	steer	no
59	pending_feedback	**reopen**	no
58	revision_proposed	revise_model	no
55	stale_validation	revalidate	no
50	divergence	reconcile	no
48	model_disagreement / anomaly	revise_model	no
47–44	viz_broken / viz_missing / viz_proposed	refine_viz / build_viz	no
45	validation_finding	revise	no
43	capability_gap	build_process	no
42	pending_gate	test	no
40	open_question	test	no
39	**proposed_study**	**seed**	no
38–35	viz_stale / viz_misplaced / viz_duplicated / viz_untied	refine_viz / relocate_viz	no

reopen and **proposed_study** (the seed op) are the two loop-closing kinds. `viz_untied` is behind a rollout flag, off by default.

closure() — a boolean over three kinds

`closed = (no unaddressed_criterion) and (no narrative_gap) and (no open_challenge).` No time, no threshold, no ratio. An investigation can be "Closed" with many open (non-blocking) items — closure counts only the three blocking kinds.

opportunities() / next_action() → execute_plan()

`next_action()` returns the single highest *value-scored* item (re-shaped as a frontier item), so the recommended next move and its consequence name *one* item. `opportunities()` re-ranks the frontier by a transparent score:

$$\text{score} = 0.30\,\text{info} + 0.40\,\text{impact} + 0.20\,\text{enable} + 0.35\,\text{closure} + 0.50\,\text{discriminate} - 0.30\,\text{cost}.$$

`execute_plan()` attaches an **executor** and a **consequence tier** to each ranked item — it runs nothing:

op	executor	consequence	auto?
revalidate	snapshot-validations	`safe`	yes
test	run-study	`simulation`	no
branch	seed-from-followup + run-study	`simulation+study`	no
seed	promote-seed	`authoring`	no
reopen	reconcile-and-reopen	`reopen`	no
reconcile	set-verdict	`review`	no
revise / revise_model	act-on-validator / discover revise	`review`	no
build_process	discover gap	`review`	no
build_viz / refine_viz (resolvable)	render-viz	`render`	no
build_viz / refine_viz (stub) / relocate_viz	discover viz	`review`	no
steer	expert (no command)	`human-only`	no
synthesize	author finding + resolve	`human-only`	no

AutonomyPolicy (0.4.0) — the machine-vs-human gate. At run time a single `AutonomyPolicy` decides, per consequence tier, what may run without a human: `safe` always runs; `simulation`/`simulation+study` run only in autonomous mode on a run-ready study; `render` is autonomous-only and resolvable-only; `authoring`, `reopen`, `review` and `build_process` go to an AI-decider path that records a rationale'd resolution; `steer`/`synthesize` are always human-only. One policy now gates *every* consequential op — the banner's "runnable" count and what Advance actually dispatches are the same set by construction.

Discharge & debts. A `resolution {kind, target, rationale, by?}` removes the matching `(kind, target)` from the frontier. A `pending_gate` (a criterion whose study verdict is supported but whose named behavior never ran) does not block closure but stays on the frontier, so the debt stays visible — distinguishing "we concluded X" from "we ran the named check for X." `capability_gap` is deliberately excluded from the discharge kinds — it closes by its own build/reject lifecycle. Advisory-only: `diagnose()` (bottleneck classifier) and `invariants()` (provenance + epistemic-separation violations) inform, they do not gate.

6 What mutates vs. what is read-only

A plain `run_cycle` is pure. State changes only through explicit, separately-triggered write-backs.

function	class	trigger
`run_cycle` (+ suggest, propose_viz)	READ-ONLY	every panel cycle
A–F (reconcile, compare, delta, discoveries)	READ-ONLY	panel steps A–F
B/C/D (plan, design, build_catalog)	READ-ONLY	panel steps B/C/D
`select_for_launch`	READ-ONLY (now wired)	the `AutonomyPolicy` gate
`build_seeded_study` / `refine_study`	READ-ONLY (pure spec)	execute + advance consult `--write`, or autonomous `write_child_study`
`compute_outcomes`	**writes** `computed_outcomes`	reconcile sync (CLI)
`advance_cycle`	dry by default; **writes** with `autonomous`	runs sims, promotes seeds, builds, reopens
`persist_cycle`	**writes** `cycle_state` + `discoveries` + caches	CLI cycle
`set_verdict` / `record_resolution`	**writes** verdict / resolutions	CLI, or AI-decider
`accept_viz` / `accept_capability_gap`	**writes** viz / gap status	CLI / panel
`reconcile_and_reopen`	**writes** reopened spec + `lifecycle_history`	CLI `ingest-feedback`

In the panel, the "Apply to investigation" button is the one surface that writes real files; "Run continuously" (advance) always runs **sandboxed**. Human-only (steer/synthesize) items are never auto-resolved.

7 Authored vs. derived — the consolidated matrix

Left = the authored substrate (the only true inputs); right = everything the engine computes from it. The one *measured* input is the simulation run.

authored by a human or AI	derived mechanically by the engine
Study **claim** sentence	claim verdict (from run outcomes), `diverges` flag
Behavior tests + `pass_if` + `measure`	per-test PASS/FAIL → study verdict
Hypotheses + numeric `predictions`	support log, favored, designs, disagreements
Acceptance criteria	`unaddressed_criterion` / `pending_gate` items
Epistemic `relation` tags	argument edges, narrative order, audit gaps
`plan_gates` (synthesis spine)	rolled-up gate verdicts
`decisions_needed`	`open_decision` items, steer actions, enable score
Findings `statement` + `from_test`	verified / provisional / contradicted classification
`required_mechanisms` contract	capability-gap proposals + build_process items
`followup_proposals` / `model_change`	proposal list, candidate actions, portfolio order
A **refuted** finding (its evidence)	**Discovery** records; **EvidenceUpdate** (revise-model / -hypothesis)
Scientist **reopening feedback**	`reopen` op; re-entered spec + `lifecycle_history`
A decision's resolution (rationale)	frontier discharge; executive renewal
The **simulation itself** (external runner)	run records → run ids, outcomes, evidence status

The engine's value is that it keeps the right column consistent with — and always re-derivable from — the left ("authored wins, derived fills"), and explicitly refuses to fabricate verdicts or resolve human-only decisions.

8 What is real vs. conditional vs. unwired

real = computes genuinely over real data. **conditional** = real only when the right fields are authored. **unwired** = engine-complete but nothing in the product calls or executes it.

stage / family	grade	why
Inputs, Core, A, B	**real**	pure readers / projection over real YAML + run data
Grading (`compute_outcomes`)	**real**	the one run-data path; feeds the verdict computed-first
F · Evaluate	**conditional**	`unresolved` without authored hypotheses + predictions
F · Discovery + model-error	**real**	fire on a *computed* refutation; 0 discoveries if a study has no hypothesis snapshot
D · Model build	**real**	novel mechanisms route to BUILD; autonomous advance builds a real ModelVersion
C · Design	**conditional**	`None` without ≥ 2 disjoint-band hypotheses
Growth (propose / generative loop)	**real**	a refuted finding proposes *and* (if authorized) runs the next study
Growth · select / AutonomyPolicy	**real**	one policy gates every consequential op; drives promotion
Growth · seed / refine	**real**	pure spec; autonomous write-back via `write_child_study`
E · Execute / advance	**real**, dry by default	sandboxed in the panel; one AutonomyPolicy gates every op
Reopen	**real**	structured re-entry on staged feedback; typed, schema-validated
Gate system	**real**	deterministic; three blocking kinds; value-ranked next action
Self-contained CLI + linkage	**real**	report / lint / observations / linkage as headless `discover` verbs (0.4.1–0.5.0)
Versioned evidence snapshot	**real**	snapshot provenance + stale-Discovery gate (0.6.0)
Viz loop	**real**	engine + full panel wiring (propose / build / declare / accept)
capability-gap bridge	**conditional**	engine-complete; no workbench endpoint yet
viz-appropriateness	thin	items only; the relocate op has no executor yet

Honest edges. Two things are defined-but-unwired, and worth naming rather than hiding: the **capability-gap bridge** is engine-complete but has no workbench endpoint (it surfaces as an unlabeled row), and **viz-appropriateness** emits items but its relocate op has no executor. The Growth autonomy gate, by contrast, is *now wired* — one `AutonomyPolicy` drives promotion in `execute_action` and `advance_cycle`. Everything else on the spine — load, project, reconcile, grade, evaluate, record, and the explicit write-backs — operates on real artifacts, and the loop closes end to end: a refuted finding proposes *and runs* the next study, a settled+verified verdict records a discovery (re-baselined if the evidence snapshot is stale), and scientist feedback re-enters through reopen.